

JAVA

07

Spring Boot

Build production-ready REST APIs and full-stack apps the way real companies actually ship them.

DURATION	LEVEL	MODULE	UPDATED
5 Sessions	Intermediate	Module 7	June 2026

IN THIS MODULE

01	Introduction to Spring Boot
02	REST API Development
03	Connecting Spring Boot with Database
04	Security & Validation
05	Integration, Testing & Postman Demo

SESSION

01

60 MIN · FRAMEWORK BASICS

Introduction to Spring Boot

◆ Learning Objectives

- Explain why Spring Boot was created and how it improves on Spring Core
- Set up a new Spring Boot project using Spring Initializr in under two minutes
- Understand what auto-configuration and embedded Tomcat actually do under the hood
- Navigate the default project folder structure and explain each directory's role
- Write and run a working Spring Boot application with a REST endpoint
- Identify the six most important Spring Boot annotations used in every project

01 The Java Web Evolution: Servlets -> Spring Core -> Spring Boot

In the early 2000s, Indian IT companies like TCS and Infosys shipped Java web apps using raw Servlets — hundreds of lines of boilerplate just to process a single login form. Spring Framework (2003) reduced that pain but still demanded heavy XML configuration files for every bean. Spring Boot, released in 2014, ended XML configuration entirely with one guiding principle: convention over configuration. A team in Ahmedabad today can have a production-ready REST server live in under an hour — no XML, no external server installation, no war deployment.

Spring Core

Requires explicit XML or Java config for every bean. Needs a separately installed and configured Tomcat or JBoss server. Significant boilerplate setup before writing any real business logic. Maximum control — but high startup friction.

Spring Boot

Auto-configures beans based on what is on the classpath. Ships with embedded Tomcat — run the JAR directly. Starter dependencies bundle all related libraries. Opinionated defaults that you can override at any time.

CONCEPT — Auto-Configuration

When Spring Boot starts, it scans the classpath. If it finds the MySQL driver JAR, it auto-configures a DataSource. If spring-web is present, it configures a DispatcherServlet. You write the code that makes your app unique — Spring Boot wires everything else. Think of it like a Gujarati thali at a dhaba: the rotli, dal, and chhaas arrive automatically on the plate. You only choose which shaak you want.

02 Spring Initializr — From Zero to Project in 60 Seconds

Spring Initializr at start.spring.io is a web form that generates a complete Maven or Gradle project. Choose your Java version (use 17 LTS), pick starter dependencies, click Generate, and a ZIP downloads instantly. Import it into IntelliJ IDEA and your project structure is fully ready — no manual pom.xml editing, no directory creation by hand. The entire setup process takes under two minutes.

<code>spring-boot-starter-web</code>	Adds REST support, Jackson JSON library, and embedded Tomcat
<code>spring-boot-starter-data-jpa</code>	Adds Spring Data JPA and Hibernate ORM for database operations
<code>mysql-connector-j</code>	JDBC driver for MySQL — needed to connect to any MySQL database
<code>spring-boot-starter-validation</code>	Bean Validation API — enables <code>@NotNull</code> , <code>@Size</code> , <code>@Email</code> annotations
<code>spring-boot-starter-security</code>	Spring Security — adds authentication and authorization support
<code>spring-boot-devtools</code>	Auto-restarts the app on code save during local development only

TRY THIS LIVE

Open start.spring.io right now. Select Maven, Java 17, and add only the 'Spring Web' dependency. Click Generate and import the ZIP into IntelliJ IDEA. Find the file with `@SpringBootApplication`. Run it and watch embedded Tomcat start in the console on port 8080. Your server is live — before you have written a single line of business logic.

03 Project Structure and Your First REST Endpoint

Spring Boot projects follow a consistent folder layout. All Java source lives under `src/main/java`, organised by package. Configuration files — including the critical `application.properties` — go in `src/main/resources`. Tests live in `src/test/java`. The main class sits at the root of your package tree annotated with `@SpringBootApplication`, which combines `@Configuration`, `@ComponentScan`, and `@EnableAutoConfiguration` into one powerful annotation.

`@SpringBootApplication`

```
public class CampusConnectApp {
    public static void main(String[] args) {
        SpringApplication.run(CampusConnectApp.class, args);
    }
}
```

Entry point — triggers auto-config, component scanning, and bean registration

Main application class — one per project

Standard Java entry point

Boots the Spring container and starts embedded Tomcat on port 8080

`@RestController`

```
public class HelloController {

    @GetMapping("/hello")
    public String hello() {

        return "Hello, Campus Connect!";
    }
}
```

Marks this class as a REST controller — all methods return JSON by default

First test controller — can be deleted once real controllers are built

Maps HTTP GET `/hello` to this method

Return value is serialised automatically and sent as the HTTP response body

Returns a plain string — Jackson wraps it in the response

PRO TIP — Embedded Tomcat Means No WAR Deployment

Spring Boot packages Tomcat inside the final JAR. To run your app anywhere, just execute: `java -jar campusconnect.jar` — no Tomcat installation, no `web.xml`, no WAR file deployment. This is why Spring Boot apps deploy cleanly to AWS EC2, Render, or any Linux server with a single command. Your local machine and production server use the exact same run mechanism.

04 Key Spring Boot Annotations at a Glance

Annotation	Purpose	Where Used
<code>@SpringBootApplication</code>	Triggers auto-configuration, component scan, and bean registration	Main class only
<code>@RestController</code>	Marks a REST API controller — every method returns JSON by default	Controller classes
<code>@GetMapping</code> / <code>@PostMapping</code>	Maps HTTP GET or POST requests to a specific method	Controller methods
<code>@Autowired</code>	Injects a Spring-managed bean automatically — prefer constructor injection	Fields or constructors
<code>@Service</code>	Marks a class as the business logic layer and registers it as a Spring bean	Service classes
<code>@Repository</code>	Marks the data access layer — also enables Spring's DB exception translation	Repository / DAO classes
<code>@Entity</code>	Maps a Java class to a database table via JPA / Hibernate	Model or domain classes
<code>@Value</code>	Injects a value from <code>application.properties</code> directly into a field	Any Spring-managed bean

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming in Java — Week 1, Lecture 1: Introduction to Java Programming Environment (covers JVM and classpath internals that Spring Boot builds on)

Link — <https://nptel.ac.in/courses/106105191>

Always link the official channel — beware fan-made or re-upload copies.

WHERE TO GO NEXT

You now have a running Spring Boot project — next session you add real REST endpoints that accept parameters and return structured JSON data.

Explore deeper — Spring Boot Reference Documentation — 'Getting Started' and 'Auto-configuration' sections at docs.spring.io/spring-boot/docs/current/reference/html

Try on your own — Modify HelloController to return a Java Map with keys 'name', 'city', and 'role' — observe how Spring Boot serialises it to a JSON object automatically

Connect the dots — Session 02 covers @RestController, @PathVariable, @RequestBody, ResponseEntity, and building the complete CRUD /events API for Campus Connect

Going further — Study Spring Boot's condition annotations — @ConditionalOnClass and @ConditionalOnMissingBean — these are the exact internal mechanisms behind every auto-configuration decision

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Spring Boot trainer. Simple Hinglish mein samjhao: Spring Boot mein auto-configuration kaise kaam karta hai? Ek example do jaise Ahmedabad ka ek naya startup apna pehla REST server banata hai bina koi XML likhe. 150 words se kam rakhna.

SESSION

02

60 MIN · API DESIGN

REST API Development

◆ Learning Objectives

- Define REST and explain its core constraints in plain language
- Map CRUD operations to the correct HTTP methods — GET, POST, PUT, DELETE
- Build REST endpoints using `@RestController`, `@GetMapping`, `@PostMapping`, `@PutMapping`, `@DeleteMapping`
- Use `@PathVariable`, `@RequestParam`, and `@RequestBody` correctly in controller methods
- Return proper HTTP status codes using `ResponseEntity`
- Build the complete `/events` CRUD API for the Campus Connect Portal

01 REST Principles and HTTP Methods

REST (Representational State Transfer) is the architectural style behind nearly every modern Indian API — Razorpay's payment gateway, Zomato's restaurant search, and IRCTC's booking system all follow REST conventions. The core idea: a resource such as an 'event' in Campus Connect has a URL, and you change its state by sending the correct HTTP method to that URL. You do not invent action-based URLs like `/createEvent` or `/deleteUser`. The URL names the thing; the HTTP method names the action.

HTTP Method	CRUD Operation	Campus Connect Example
GET	Read — fetch data without modifying anything	GET <code>/api/events</code> — returns all events as a JSON array
POST	Create — add a brand new resource	POST <code>/api/events</code> — creates a new event from the JSON body
PUT	Update — fully replace an existing resource	PUT <code>/api/events/5</code> — replaces entire event with id 5
DELETE	Delete — permanently remove a resource	DELETE <code>/api/events/5</code> — deletes event with id 5
PATCH	Partial Update — change specific fields only	PATCH <code>/api/events/5</code> — updates only the title or date

CONCEPT — `@RestController`

`@RestController` is a shortcut combining `@Controller` and `@ResponseBody`. Any class annotated with `@RestController` automatically converts every method's return value to JSON using the Jackson library — no extra configuration needed. In contrast, a plain `@Controller` returns a view name (like a JSP page). Think of `@RestController` as a Swiggy delivery partner who always packs food in the same standardised box format — consistent, predictable JSON output every time.

02 Building REST Endpoints

`@RestController`

This class returns JSON for every method — no view resolver involved

`@RequestMapping("/api/events")`

Base path prefix applied to all endpoints in this controller

`public class EventController {`

`@GetMapping`

Handles HTTP GET /api/events — returns all events as a JSON array

`public List<Event> getAllEvents() {`

List<Event> is automatically serialised to a JSON array by Jackson

`return eventService.findAll();`

Controller only handles HTTP concerns — business logic stays in the service

`}`

`@GetMapping("/{id}")`

Handles HTTP GET /api/events/5 — fetches one event by its id

`public ResponseEntity<Event> getById(@PathVariable Long id) {`

`ResponseEntity` gives full control over the HTTP status code and body

`return eventService.findById(id)`

`.map(ResponseEntity::ok)`

Returns 200 OK with the event body if the event exists

`.orElse(ResponseEntity.notFound().build());`

Returns 404 Not Found if no event matches the id

`}`

`}`

TRY THIS LIVE

Add this EventController with a hardcoded list — no database yet. Run the app and open

<http://localhost:8080/api/events> in your browser. You will see a JSON array. Now change `@GetMapping` to

`@PostMapping` and refresh the browser — you will get a 405 Method Not Allowed error. That is correct REST behaviour. It also demonstrates exactly why Postman exists.

03 @PathVariable, @RequestParam, and @RequestBody

◆ Three Ways to Receive Data in a REST Method

Spring Boot provides three different annotations to receive data in a REST endpoint. `@PathVariable` reads a value embedded in the URL path, like the '5' in /events/5. `@RequestParam` reads a query string parameter, like 'city' in /events?city=Ahmedabad. `@RequestBody` reads the full JSON payload sent in the HTTP request body — used for POST and PUT where the client sends an entire object. Mixing these up is the most common beginner mistake in

Spring Boot REST development.

```
// @PathVariable – value is embedded directly in the URL path
// Best for resource identifiers like numeric IDs
@GetMapping("/{id}")
public Event getById(@PathVariable Long id) { ... }
// URL /api/events/42 binds 42 to the id parameter automatically

// @RequestParam – value is a URL query string parameter
// Best for optional filters, sorting, or pagination controls
@GetMapping("/search")
public List<Event> search(@RequestParam String city) { ... }
// URL /api/events/search?city=Surat binds 'Surat' to city

// @RequestBody – value is the full JSON payload in the request body
// Required for POST and PUT — client sends a JSON object
@PostMapping
public Event create(@RequestBody EventDTO dto) { ... }
// Jackson automatically maps the incoming JSON body to EventDTO fields
```

PRO TIP — Always Use a DTO, Not the Entity Directly

Never expose your `@Entity` class as a `@RequestBody` parameter in production. Create a separate `EventDTO` (Data Transfer Object) class for incoming API requests. This prevents mass-assignment attacks where a malicious client sends unexpected fields like `'adminFlag: true'`, and it keeps your API contract independent from your database schema — two things that legitimately need to change at different times.

04

Campus Connect /events — Full CRUD API

```
@PostMapping
// Handles HTTP POST /api/events — creates a new event
public ResponseEntity<Event> create(@RequestBody EventDTO dto) {
    // @RequestBody maps the incoming JSON payload to an EventDTO object
    Event saved = eventService.create(dto); // All creation logic lives in the service layer — not here
    return ResponseEntity.status(HttpStatus.CREATED).body(saved);
    // Returns HTTP 201 Created — the correct status code for resource creation
}

@PutMapping("/{id}")
// Handles HTTP PUT /api/events/5 — fully replaces event with id 5
```

```
public ResponseEntity<Event> update(@PathVariable Long id, @RequestBody EventDTO dto)
{
    Combines @PathVariable for the ID and @RequestBody for the replacement data
    return ResponseEntity.ok(eventService.update(id, dto));
    Returns HTTP 200 OK with the updated event object in the body
}

@DeleteMapping("/{id}")
Handles HTTP DELETE /api/events/5 — removes
event with id 5

public ResponseEntity<Void> delete(@PathVariable Long id) {
    Void return type means no body is sent in the response
    eventService.delete(id);
    return ResponseEntity.noContent().build();
    Returns HTTP 204 No Content — the correct code for
    successful delete operations
}
```

CASE STUDY — Razorpay REST API Design

Context — Razorpay, a Bengaluru-based fintech unicorn, processes billions of rupees in payments for merchants across India — from Ahmedabad textile exporters to large enterprise e-commerce platforms.

Challenge — Developer teams with very different skill levels needed to integrate Razorpay payments in days, not weeks. An inconsistent API would directly cost Razorpay merchant sign-ups and revenue.

What they did — Razorpay designed their entire developer API around strict REST conventions: POST /orders to create, GET /orders/:id to fetch one, clean JSON request and response bodies, and standard HTTP status codes (200, 201, 400, 404, 500) for every outcome — the same pattern you are building in Campus Connect.

Result — Developers across India can integrate Razorpay in a single day because the API is entirely predictable. If you understand REST conventions, you understand the Razorpay API without reading more than the quick-start guide.

Takeaway — REST conventions are not academic theory — they are what makes APIs that real developers actually enjoy integrating.

Source: Razorpay Developer Documentation, razorpay.com/docs/api

WHERE TO GO NEXT

Your REST controller is complete — next session you wire these endpoints to a real MySQL database using Spring Data JPA.

Explore deeper — @ExceptionHandler and @ControllerAdvice annotations — for building a centralised error handler that returns consistent JSON error responses across your whole API

Try on your own — Add a GET /api/events/search?title=keyword endpoint to your hardcoded list that filters events by title using Java streams

Connect the dots — Session 03 introduces @Entity, JpaRepository, application.properties database config, and the full Entity -> Repository -> Service -> Controller architecture

Going further — Read about HTTP idempotency — why GET, PUT, and DELETE are idempotent but POST is not, and why this matters when a mobile client retries a dropped network request

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Spring Boot REST API expert. Simple Hinglish mein samjhao: @PathVariable, @RequestParam, aur @RequestBody mein kya fark hai? Ek example lo jaise Zomato ke restaurant search API ka. 150 words se kam rakhna.

SESSION

03

60 MIN · DATA LAYER

Connecting Spring Boot with Database

◆ Learning Objectives

- Explain what Spring Data JPA is and how it eliminates JDBC boilerplate
- Configure a MySQL database connection in application.properties
- Create a JPA @Entity class that maps to a database table with correct annotations
- Define a JpaRepository interface and use its built-in CRUD methods without writing SQL
- Build the complete Entity -> Repository -> Service -> Controller layered architecture
- Perform Create, Read, Update, and Delete operations on the events table

01 Why Spring Data JPA?

In Module 2 (JDBC), you wrote every SQL query manually — getConnection(), prepareStatement(), executeQuery(), and then iterated ResultSet row by row. For a project with 10 tables, that becomes thousands of lines of nearly identical boilerplate. Spring Data JPA wraps Hibernate (an ORM) to abstract all of this. You define a Java class that maps to a database table, extend one interface, and Spring generates all the SQL for you at runtime. Indian fintech companies like Juspay and Razorpay use Spring Data JPA for exactly this reason — developers focus on business logic, not SQL plumbing.

Plain JDBC (Manual)

Write PreparedStatement for every query. Manually map each ResultSet column to a Java field. Handle connection open and close explicitly. Any schema change means updating multiple query strings. High control, but extremely verbose.

Spring Data JPA

Annotate the Java class — Spring generates SQL automatically. Built-in methods: findAll(), findById(), save(), delete(). Derived query methods like findByCity() with zero SQL. Schema changes only require updating the @Entity class.

CONCEPT — The JpaRepository Pattern

Spring Data JPA follows the Repository pattern. You create an interface that extends JpaRepository<EntityClass, IdType>. Spring Boot sees this interface at startup and generates a complete implementation — including findAll(), findById(), save(), deleteById(), and more — without you writing a single line of implementation code. It is the closest thing to magic that Java developers encounter in production.

02 Database Configuration in application.properties

Spring Boot reads all database configuration from src/main/resources/application.properties. You specify the JDBC URL, credentials, driver class, and Hibernate dialect here. The property spring.jpa.hibernate.ddl-auto controls how Hibernate handles your schema at startup — use 'update' in development (it creates or modifies tables to match your

entities) and 'validate' in production (it checks but never changes the schema).

```
# MySQL datasource configuration
spring.datasource.url=jdbc:mysql://localhost:3306/campus_connect
    JDBC connection string — replace campus_connect with your database name
spring.datasource.username=root
    Database username — use a restricted user in production, never root
spring.datasource.password=your_password_here
    Never commit real passwords to Git — use environment variables in production
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
    MySQL 8 JDBC driver class — included via mysql-connector-j dependency

# JPA / Hibernate settings
spring.jpa.hibernate.ddl-auto=update
    Auto-creates or updates tables to match @Entity classes — use 'validate' in production
spring.jpa.show-sql=true
    Prints every SQL query to the console — helpful during development
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
    Tells Hibernate to generate SQL syntax compatible with MySQL 8
```

TRY THIS LIVE

Add spring-boot-starter-data-jpa and mysql-connector-j to your pom.xml, paste the application.properties block above with your local MySQL credentials, and restart the app. Watch the console: if you see 'HikariPool-1 - Start completed' it means the database connection pool is active. If you see 'Connection refused', your MySQL service is not running — start it from MySQL Workbench or run: `sudo service mysql start`.

03 Entity, Repository, and Service Layers

Spring Boot applications use a strict four-layer architecture. The `@Entity` class maps a Java object to a database row. The `@Repository` interface extends `JpaRepository` and provides database access methods. The `@Service` class holds business logic and calls the repository. The `@RestController` handles HTTP and calls the service. No layer should skip a level — controllers must never call repositories directly. This separation makes each layer independently testable and maintainable.

```
@Entity
@Table(name = "events")

public class Event {

    Marks this class as a JPA entity — Hibernate maps it to a database table
    Explicitly names the table 'events' in MySQL — good practice to be explicit
```

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;

@Column(nullable = false)
private String title;

private String description;

private LocalDate eventDate;

private String location;

// getters and setters required by JPA
}
```

Marks this field as the primary key column

Uses MySQL AUTO_INCREMENT — database assigns the id on insert

Always use Long (wrapper type), not long, for nullable JPA IDs

Maps to a NOT NULL column in MySQL — Spring also uses this for validation

Columns without @Column annotation use defaults — nullable, no length limit

JPA maps LocalDate to a MySQL DATE column automatically

City or venue — e.g. 'Ahmedabad', 'IIM Ahmedabad Campus'

```
@Repository
public interface EventRepository extends JpaRepository<Event, Long> {
    List<Event> findByLocation(String location);
    List<Event> findByTitleContaining(String keyword);
}

@Service
public class EventService {
    @Autowired
    private EventRepository eventRepository;

    public List<Event> findAll() { return eventRepository.findAll(); }
    public Optional<Event> findById(Long id) { return eventRepository.findById(id); }
    public Event save(Event event) { return eventRepository.save(event); }
    public void delete(Long id) { eventRepository.deleteById(id); }
}
```

Marks this as a data access layer bean — also enables Spring's DB exception translation

Spring generates full CRUD implementation at startup — no SQL code needed

Derived query — Spring translates this to: SELECT * FROM events WHERE location = ?

Derived query — translates to: SELECT * FROM events WHERE title LIKE %keyword%

Business logic layer — never put database calls directly in the controller

Spring injects the EventRepository implementation automatically

Fetches all events — no SQL needed

Returns Optional — handles the 'event not found' case cleanly

save() handles both INSERT (new entity) and UPDATE (existing entity)

Throws EmptyResultDataAccessException if id not found — handle in controller

```
}
```

PRO TIP — Always Annotate Service Methods with @Transactional

Any service method that writes to the database — save, update, delete — should be annotated with `@Transactional`. This ensures the entire method runs as a single database transaction: if any step fails, all changes roll back automatically. Without `@Transactional`, a partial failure (e.g. saving the event but failing to update related records) leaves your database in an inconsistent state. Add `@Transactional` at the class level for service classes where most methods write to the DB.

04 Spring Data JPA Annotations Quick Reference

Annotation	Purpose	Example
<code>@Entity</code>	Maps a Java class to a database table	<code>@Entity public class Event { }</code>
<code>@Table(name=...)</code>	Explicitly sets the table name in the database	<code>@Table(name = "events")</code>
<code>@Id</code>	Marks the primary key field	<code>@Id private Long id;</code>
<code>@GeneratedValue</code>	Configures how the primary key is auto-generated	<code>@GeneratedValue(strategy = GenerationType.IDENTITY)</code>
<code>@Column</code>	Controls column constraints like nullability and length	<code>@Column(nullable = false, length = 100)</code>
<code>@Transient</code>	Excludes a field from database mapping — JPA ignores it	<code>@Transient private String tempFlag;</code>
<code>@OneToMany</code> / <code>@ManyToOne</code>	Declares relationships between entities	<code>@OneToMany(mappedBy = "event")</code>
<code>@JoinColumn</code>	Specifies the foreign key column for a relationship	<code>@JoinColumn(name = "event_id")</code>

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming in Java — Week 8: JDBC and Database Connectivity (covers the low-level mechanics that Spring Data JPA abstracts away)

Link — <https://nptel.ac.in/courses/106105191>

WHERE TO GO NEXT

Your data layer is complete — next session you add Spring Security to protect write operations and Bean Validation to reject invalid input.

Explore deeper — Spring Data JPA derived query methods documentation — learn the method naming convention that lets you write `findByLocationAndEventDateAfter(String, LocalDate)` with no SQL

Try on your own — Add a `findByLocation(String city)` method to `EventRepository` and call it from a new GET `/api/events/by-city?city=Ahmedabad` endpoint

Connect the dots — Session 04 adds Spring Security (Basic Auth for protected endpoints) and Bean Validation (`@NotNull`, `@Size`, `@Email`) on the `EventDTO`

Going further — Study JPQL (Java Persistence Query Language) and the `@Query` annotation — for complex queries that cannot be expressed as derived method names

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Spring Data JPA trainer. Simple Hinglish mein explain karo: JpaRepository kya hai aur ye JDBC se better kyun hai? Ek example lo jaise ek Surat-based events management app ka database layer banana. 150 words se kam rakhna.

SESSION

04

60 MIN · API PROTECTION

Security & Validation

◆ Learning Objectives

- Explain why API security is mandatory even for internal and student projects
- Add Spring Security to a Spring Boot project and understand what it auto-configures
- Protect specific endpoints using Basic Auth and a SecurityFilterChain configuration
- Configure a UserDetailsService with in-memory credentials for development
- Apply Bean Validation annotations to DTO fields — @NotBlank, @Size, @Email, @NotNull
- Trigger validation with @Valid in a controller and return structured 400 error responses

01 Why API Security Is Non-Negotiable

Without security, any endpoint you expose is publicly accessible — anyone with a browser or Postman can POST fake events, DELETE real data, or flood your API with junk. In India, unsecured APIs have led to several high-profile data leaks including the exposure of personal records from government portals and UPI apps. The OWASP API Security Top 10 project was specifically created because broken authentication and missing authorisation are consistently the top causes of API breaches globally. Adding even Basic Auth to your Campus Connect API immediately eliminates the majority of opportunistic attacks.

OF WEB TRAFFIC IS NOW API CALLS

83%

Source: Akamai State of the Internet Report, 2022 — making unsecured APIs the largest attack surface in modern applications

CONCEPT — Spring Security Auto-Configuration

The moment you add spring-boot-starter-security to pom.xml and restart, Spring Boot auto-configures a default security layer: every endpoint is immediately protected, a random password is printed to the console, and a login form appears in the browser. This is intentional — Spring Security defaults to 'deny all' so that developers must explicitly decide what to allow. You then override this by providing your own SecurityFilterChain bean.

02 Configuring Basic Auth and Protecting Endpoints

@Configuration

Marks this as a Spring configuration class — beans defined here are auto-registered

@EnableWebSecurity

Activates Spring Security's web security support

```
public class SecurityConfig {
```

@Bean

Spring picks up this SecurityFilterChain bean and uses it as the security configuration

```
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
```

```
    http
```

```
        .authorizeHttpRequests(auth -> auth
```

Define which endpoints need authentication

```
        .requestMatchers(HttpMethod.GET, "/api/events").permitAll()
```

Public: anyone can read the events list without logging in

```
        .anyRequest().authenticated()
```

All other requests (POST, PUT, DELETE) require valid credentials

```
    )
```

```
        .httpBasic(Customizer.withDefaults())
```

Enables HTTP Basic Auth — Postman or React sends credentials in the header

```
        .csrf(csrf -> csrf.disable());
```

Disable CSRF for REST APIs — CSRF is for browser-based form submissions only

```
    return http.build();
```

```
}
```

@Bean

```
public UserDetailsService userDetailsService() {
```

Defines who is allowed to authenticate — use a database-backed service in production

```
UserDetails admin = User.withDefaultPasswordEncoder()
```

withDefaultPasswordEncoder() is for development only — use BCrypt in production

```
    .username("admin")
```

Login username

```
    .password("campus@123")
```

Login password — store this in environment variables, not in code

```
    .roles("ADMIN")
```

Grants the ROLE_ADMIN authority to this user

```
    .build();
```

```
return new InMemoryUserDetailsManager(admin);
```

In-memory store — suitable for development and demos only

```
}
```

```
}
```

TRY THIS LIVE

Add spring-boot-starter-security to pom.xml and restart the app. Try calling GET /api/events in Postman without any auth — you should get 200 OK. Now try POST /api/events without auth — you should get 401 Unauthorized. Then add Basic Auth in Postman (username: admin, password: campus@123) and POST again — you should get 201 Created. This is your API protected.

WARNING — Never Hardcode Credentials in Source Code

The username and password above are acceptable only for a classroom demo. In any real project — even a student portfolio on GitHub — never commit credentials to your repository. Use Spring Boot's environment variable injection: `spring.datasource.password=${DB_PASSWORD}`. Set the variable in your OS or deployment platform. A single public GitHub commit with hardcoded credentials is enough for automated bots to find and exploit your database.

03 Input Validation with Bean Validation API

A REST API must never trust what the client sends. Without server-side validation, a client can POST an event with a null title, a date in the past, or a description containing SQL injection. Bean Validation (Jakarta Validation API) lets you declare constraints directly on DTO fields using annotations. Spring Boot then checks these automatically when the `@Valid` annotation is present on a `@RequestBody` parameter — no manual if-else validation code needed.

```
public class EventDTO {
```

DTO — the object that arrives from the client in the HTTP request body

```
    @NotBlank(message = "Event title is required")
    @NotBlank rejects null, empty string, and whitespace-only strings
    @Size(min = 3, max = 100, message = "Title must be between 3 and 100 characters")
    Length constraint — prevents both one-character titles and huge text dumps
    private String title;

    @NotNull(message = "Event date is required")
    @NotNull rejects null — use @NotBlank for String fields, @NotNull for objects
    @FutureOrPresent(message = "Event date cannot be in the past")
    Rejects dates before today — prevents creating events for dates already gone
    private LocalDate eventDate;

    @NotBlank(message = "Location is required")
    private String location;
```

City or venue name — e.g. 'Ahmedabad', 'GNLU Campus, Gandhinagar'

```
    @Email(message = "Organiser email must be a valid email address")
    @Email validates format — rejects 'notanemail' or 'missing@'
    private String organiserEmail;
}
```

```
@PostMapping
public ResponseEntity<Event> create(@Valid @RequestBody EventDTO dto) {
    @Valid triggers Bean Validation — Spring checks all constraints before executing the method
```

```

Event saved = eventService.create(dto);
return ResponseEntity.status(HttpStatus.CREATED).body(saved);
}

// If @Valid fails, Spring Boot auto-returns HTTP 400 Bad Request
Add @ControllerAdvice to customise the error response format (see Going Further)
// Default error response includes field name and the message from your annotation
e.g. { "title": "Event title is required", "eventDate": "Event date is required" }

```

This line only runs if ALL @Valid constraints pass — no extra if-else needed

PRO TIP — Add a Global Exception Handler with @ControllerAdvice

By default, Spring Boot's validation error response is a deeply nested JSON object that is hard to read. Create a class annotated with @ControllerAdvice and a method with @ExceptionHandler(MethodArgumentNotValidException.class). Inside, extract the field errors and return a clean Map of fieldName -> errorMessage with HTTP 400. This makes your API's error responses as predictable as its success responses.

04 Security and Validation Reference

Basic Auth (Used Here)

Credentials sent in every request header. Simple to configure and test in Postman. Stateless — no server-side session. Use only over HTTPS in production. Works well for internal APIs, admin dashboards, and learning projects.

JWT (JSON Web Tokens — Next Level)

Client logs in once and receives a signed token. Token is sent in subsequent requests — no credentials on every call. Token carries user roles and expiry. Industry standard for React/mobile frontends. Requires spring-boot-starter-oauth2-resource-server.

Annotation	What It Rejects	Common Use Case
@NotNull	null values only	Object fields like LocalDate, Long, Boolean
@NotBlank	null, empty string, and whitespace-only	All String fields — stricter than @NotNull
@Size(min, max)	strings or collections outside the length range	Titles, descriptions, usernames
@Email	strings that are not a valid email format	Email address fields
@Min(value) / @Max(value)	numbers below min or above max	Age, quantity, score fields

Annotation	What It Rejects	Common Use Case
@FutureOrPresent	dates before today	Event dates, booking dates
@Pattern(regex)	strings not matching a regular expression	Phone numbers, postal codes, PIN fields

WHERE TO GO NEXT

Your API is now protected and validated — the final session connects it to a React frontend, tests it with Postman, and packages it for deployment.

Explore deeper — Spring Security's @PreAuthorize annotation — for method-level security where different roles can access different endpoints or operations

Try on your own — Add an @Email validation to the organiserEmail field in EventDTO and test in Postman — send a request with an invalid email and observe the 400 response

Connect the dots — Session 05 adds CORS configuration so the React frontend can call your API, demonstrates full Postman testing, and shows how to package the app as a deployable JAR

Going further — Study Spring Boot with JWT using spring-boot-starter-oauth2-resource-server — this is the real-world standard used by Razorpay, Juspay, and most Indian product companies

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Spring Security expert. Simple Hinglish mein samjhao: Basic Authentication aur JWT mein kya fark hai? Ek Ahmedabad ka startup agar apna campus events portal bana raha ho to kaunsa use karna chahiye aur kyun? 150 words se kam rakhna.

SESSION

05

60 MIN · FULL STACK INTEGRATION

Integration, Testing & Postman Demo

◆ Learning Objectives

- Explain what CORS is and why browsers block cross-origin API calls by default
- Configure CORS in Spring Boot using `@CrossOrigin` and a global `WebMvcConfigurer`
- Test all CRUD API endpoints comprehensively using Postman collections and environments
- Write a React component that fetches and displays events from the Spring Boot API using Axios
- Trace a complete full-stack request from React UI to Spring Boot to MySQL and back
- Package the Spring Boot application as an executable JAR and understand deployment options

01 CORS — Why the Browser Blocks Your API

When your React app runs on `http://localhost:3000` and your Spring Boot API runs on `http://localhost:8080`, the browser treats them as different origins and blocks the request by default. This is the browser's Same-Origin Policy — a security feature that prevents scripts on one domain from reading responses from another. CORS (Cross-Origin Resource Sharing) is the mechanism that lets your Spring Boot API explicitly say: 'I allow requests from `localhost:3000`.' Without CORS configuration, your React frontend will receive a network error even when the backend is working perfectly.

CONCEPT — How CORS Works

When a browser makes a cross-origin request, it first sends a preflight `OPTIONS` request asking 'are you allowed to talk to me?' The server must respond with the correct `Access-Control-Allow-Origin` header. Spring Boot's CORS configuration automatically handles this preflight exchange. Important: CORS is a browser enforcement only — Postman, curl, and server-to-server calls bypass it entirely. If your API works in Postman but fails in the React browser, it is almost always a missing CORS header.

```
// Option 1 — Per-controller annotation (quick, for development)
```

```
    Add directly above the @RestController class declaration
```

```
@CrossOrigin(origins = "http://localhost:3000")
```

```
    Allows requests from your React dev server to this controller only
```

```
@RestController
```

```
@RequestMapping("/api/events")
```

```
public class EventController { ... }
```

```
// Option 2 — Global CORS config (recommended for production)
```

```
    Applies to all controllers and all endpoints in one place
```

```
@Configuration
```

```
public class CorsConfig implements WebMvcConfigurer {
```

```
    @Override
```

```
public void addCorsMappings(CorsRegistry registry) {
    registry.addMapping("/api/**")           Apply CORS rules to all paths starting with /api/
    .allowedOrigins("http://localhost:3000")
    Allow requests from React dev server — add production domain here for deployment
    .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
    List every HTTP method your frontend actually uses
    .allowedHeaders("*")                     Allow all request headers including Authorization for
                                           Basic Auth
    .allowCredentials(true);                 Required when sending Authorization headers —
                                           enables credential-bearing requests
}
```

TRY THIS LIVE

Add the global CorsConfig class to your project. Start both your Spring Boot app (port 8080) and your React app (port 3000). Open the browser Console in React and make a fetch call to `http://localhost:8080/api/events`. Without CORS config you saw a red CORS error — with the config you should now see the events data returned. If you still see a CORS error, check that `allowCredentials(true)` and `allowedOrigins` are set correctly together.

02 API Testing with Postman

Postman is the industry-standard tool for testing REST APIs before connecting a frontend. Indian companies from Freshworks in Chennai to Razorpay in Bengaluru include Postman collections in their developer onboarding. The correct testing workflow is: test each endpoint individually in Postman first, confirm all HTTP status codes match expectations (200, 201, 204, 400, 401, 404), then connect the React frontend. Never debug React and Spring Boot simultaneously — isolate each layer.

Postman Feature	Purpose	How to Use for Campus Connect
Collections	Group related requests together and save them	Create a 'Campus Connect API' collection with one request per endpoint
Environments	Store variables like base URL and auth token separately from requests	Create 'Dev' environment with <code>base_url = http://localhost:8080</code> and <code>password = campus@123</code>
Pre-request Scripts	Run JavaScript before a request executes	Auto-generate a fresh test event title using <code>Math.random()</code>

Postman Feature	Purpose	How to Use for Campus Connect
Tests tab	Write assertions that run after every request	Assert: <code>pm.response.to.have.status(201)</code> after every POST <code>/events</code>
Basic Auth tab	Set username and password for authenticated endpoints	Add admin / campus@123 under the collection-level Auth to apply to all requests
Body — raw JSON	Send a JSON payload for POST and PUT requests	Paste your EventDTO JSON here with Content-Type: application/json

PRO TIP — Save the Created ID for Follow-Up Requests

In your POST `/api/events` request, add this to the Tests tab: `pm.environment.set('lastEventId', pm.response.json().id)`. Now every GET `/api/events/{{lastEventId}}`, PUT `/api/events/{{lastEventId}}`, and DELETE `/api/events/{{lastEventId}}` request automatically uses the ID of the event you just created. This technique lets you run the full CRUD test flow in sequence without manually copying IDs between requests.

03 React + Spring Boot: End-to-End Data Flow

Axios is the standard HTTP client library in React for consuming REST APIs. When the `EventList` component mounts, it calls `axios.get()` to fetch the events from your Spring Boot backend. The response is a JavaScript array — React maps over it and renders each event as a card or list item. For POST requests, Axios sends the form data as a JSON body with the `Content-Type: application/json` header, which Spring Boot's `@RequestBody` reads automatically.

```
import axios from 'axios';
import { useState, useEffect } from 'react';

const API_BASE = 'http://localhost:8080/api';

const EventList = () => {
  const [events, setEvents] = useState([]);

  const [error, setError] = useState(null);

  useEffect(() => {
    axios.get(`${API_BASE}/events`)
```

Install Axios first: `npm install axios`

`useState` for storing events, `useEffect` for triggering the API call on mount

Extract base URL — change this once when deploying to production

State to hold the array of events returned from the backend

State to hold any error message if the API call fails

Runs once when the component mounts — equivalent to `componentDidMount`

HTTP GET `/api/events` — no auth header needed (endpoint is public)

```

        .then(res => setEvents(res.data))
        .catch(err => setError('Failed to load events. Please try again.'));
    Always handle errors — never leave a failed API call silently
    }, []);

    if (error) return <p>{error}</p>;
    Show a user-friendly error message if the fetch failed
    return events.map(e => <div key={e.id}>{e.title} — {e.location}</div>);
    Render each event — always set key={e.id} to avoid React reconciliation warnings
    };
    export default EventList;

```

CASE STUDY — Info Edge (Naukri.com) Full-Stack Architecture

Context — Info Edge, the Noida-based company behind Naukri.com, 99acres, and Jeevansathi, serves millions of job seekers and recruiters across India every day using a Java-based backend.

Challenge — Naukri's job listing pages needed to load instantly while keeping their Java business logic centralised. Mixing server-rendered JSP pages and mobile apps created duplicate logic and inconsistent user experiences.

What they did — Info Edge progressively migrated to a React frontend consuming Java REST APIs — exactly the pattern you built this module. CORS was configured at the API gateway level to allow both their web and mobile frontends to consume the same endpoints. Postman collections became part of their CI/CD pipeline to automatically verify API contracts before every deployment.

Result — Naukri can now deploy frontend and backend independently, run automated API tests on every code merge, and serve the same data to their website, Android app, and iOS app from one REST backend.

Takeaway — The Spring Boot + React architecture you built in this module is not academic — it is the same pattern running India's largest job portal.

Source: Info Edge India Limited, Engineering Blog; infoedge.in

04 JAR Packaging and Deployment Basics

When your Campus Connect API is ready to deploy, Spring Boot's Maven plugin packages everything into a single executable JAR file — your code, all dependencies, and Tomcat — in one command. This JAR can run on any machine with Java 17 installed. For cloud deployment, you simply copy the JAR to an AWS EC2 instance or a Render service and run it. The React frontend is built separately with `npm run build` and deployed to a CDN or Netlify — a completely independent deployment cycle from the backend.

```
mvn clean package -DskipTests
```

Build the executable JAR in the target/ folder — skip tests for a quick build

```
java -jar target/campus-connect-0.0.1-SNAPSHOT.jar
```

Run the packaged JAR — Tomcat starts and listens on port 8080

```
# Set environment variables before running in production
```

Never hardcode secrets in the JAR

```
export DB_URL=jdbc:mysql://prod-db-host:3306/campus_connect
```

Override application.properties values at runtime using env vars

```
export DB_USERNAME=appuser
```

```
export DB_PASSWORD=secure_prod_password
```

```
java -jar target/campus-connect-0.0.1-SNAPSHOT.jar
```

Spring Boot reads the exported env vars and uses them in database config

```
# For React frontend deployment
```

```
npm run build
```

Creates a production-optimised build in the build/ folder

```
# Upload build/ folder to Netlify, Vercel, or AWS S3 static hosting
```

Frontend is static HTML/CSS/JS — no server needed for the React files

WHERE TO GO NEXT

You have a complete, secured, and tested Spring Boot REST API connected to a React frontend — Module 7 is complete. Mini Project 10 puts all of this together.

Explore deeper — Spring Boot Actuator — add spring-boot-starter-actuator to expose /actuator/health and /actuator/metrics endpoints for monitoring your deployed app

Try on your own — Write a React form that POSTs a new event to /api/events using axios.post() with Basic Auth credentials in the request config — use Postman first to verify the endpoint works

Connect the dots — Module 8 (Spring Web Services) builds on this by integrating AI APIs — calling OpenAI and building a Campus Bot endpoint using the LangChain4J Java library

Going further — Study Docker containerisation for Spring Boot — package your JAR and MySQL together in a docker-compose.yml file so the entire stack runs with one command on any machine

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a full-stack Java developer. Simple Hinglish mein samjhao: CORS kya hota hai aur Spring Boot mein isko configure karna kyun zaroori hai jab React frontend alag port pe chalta ho? Ek Campus Connect-style example use karo. 150 words se kam rakhna.

REVIEW



INTERVIEW PREP · MODULE 7

Interview Questions

◆ Spring Boot

Rehearse each answer out loud and aim for a 60-second response. Use the hint only as a nudge if you are stuck. Compare your attempt with the model answer afterward to find gaps.

1

What is Spring Boot and how is it different from traditional Spring Core?

Think about configuration, setup time, and embedded server support.

MODEL ANSWER

Spring Boot is an opinionated layer on top of the Spring Framework that eliminates the boilerplate configuration needed to set up a Spring application. Traditional Spring requires manual configuration of DispatcherServlet, datasource, ViewResolver, and dozens of XML or Java config files. Spring Boot provides auto-configuration (sensible defaults based on what is on the classpath), starter dependencies (pre-packaged bundles of related libraries), an embedded Tomcat server (no WAR deployment needed), and a single `@SpringBootApplication` entry point. You can build and run a REST API in minutes instead of hours.

2

Why is auto-configuration considered a benefit in Spring Boot applications?

Consider how much manual configuration is needed in regular Spring.

MODEL ANSWER

Auto-configuration means Spring Boot inspects the classpath and automatically configures beans you would otherwise set up manually. If `spring-data-jpa` is on the classpath with a database driver, Spring Boot automatically configures a `DataSource`, `EntityManagerFactory`, and transaction manager. You do not write a single line of configuration for this. This dramatically reduces the time to get a working application running, reduces configuration bugs, and lets developers focus on business logic. You can always override defaults in `application.properties` when the auto-configured behaviour is not what you need.

3

What is the role of the embedded server (like Tomcat) in a Spring Boot application?**MODEL ANSWER**

Traditionally, you deploy a WAR file to an external Tomcat server. Spring Boot embeds Tomcat (or Jetty or Undertow) inside the JAR, so running `java -jar myapp.jar` starts both the web server and your application in one step. This simplifies deployment — especially in containers — since there is no separate server to install or configure. The embedded server is configured entirely from `application.properties` (port, context path, SSL). It also makes local development faster because you restart the application rather than redeploying to a server.

4

How would you create a new Spring Boot project using Spring Initializr?*Focus on the steps and options you select during project creation.***MODEL ANSWER**

Go to start.spring.io. Choose Maven (or Gradle), Java, and the Spring Boot version. Fill in Group (com.tops), Artifact (campus-connect), and packaging (JAR). Add dependencies: Spring Web (for REST), Spring Data JPA (for database), MySQL Driver, and Spring Security if needed. Click Generate to download a ZIP. Extract it, open in IntelliJ or Eclipse as a Maven project. The main class annotated `@SpringBootApplication` is already present. Add your application.properties with the DB URL and credentials, and the project is ready to run.

5

Explain the difference between GET, POST, PUT, and DELETE HTTP methods in the context of REST APIs.*Think about what each method is typically used for.***MODEL ANSWER**

GET retrieves data — safe and idempotent (calling it multiple times does not change state). POST creates a new resource — not idempotent (calling it twice creates two records). PUT updates an existing resource — idempotent (calling it twice with the same data leaves the same result). DELETE removes a resource — idempotent. Following these semantics makes your API predictable and allows HTTP clients, caches, and proxies to behave correctly. REST APIs conventionally map CRUD operations: Create->POST, Read->GET, Update->PUT, Delete->DELETE.

6

What does the `@RestController` annotation do in a Spring Boot application?**MODEL ANSWER**

`@RestController` is a combination of `@Controller` and `@ResponseBody`. `@Controller` marks the class as a Spring MVC controller. `@ResponseBody` tells Spring to serialise the return value of each method directly to the HTTP response body as JSON (using Jackson), instead of treating it as a view name to resolve. This means you can return a Java object or list from a controller method and Spring Boot automatically converts it to JSON. It is the standard annotation for building REST APIs in Spring Boot.

7

How would you map a URL path to a specific method in your controller using Spring Boot annotations?*Consider which annotation is used for mapping endpoints.***MODEL ANSWER**

Use `@GetMapping`, `@PostMapping`, `@PutMapping`, or `@DeleteMapping` on the method with the path: `'@GetMapping("/events")'` maps GET /events to that method. You can set a base path on the class with `'@RequestMapping("/api/v1")'` and let method annotations refine it further. `@GetMapping("/events/{id}")` maps GET /events/5 to the method, where the path variable is extracted with `@PathVariable`. These are shorthand for the more verbose `@RequestMapping` with method attribute.

8

In a REST API, how do you extract a value from the URL path (for example, /events/5) inside a controller method?

Think about which annotation helps bind URL segments to method parameters.

MODEL ANSWER

`@GetMapping("/events/{id}")` public Event getEvent(@PathVariable Long id) { return eventService.findById(id); }. `@PathVariable` binds the `{id}` template variable in the URL to the method parameter. The parameter name must match the template variable name (or use `@PathVariable("id")` if it differs). Spring automatically converts the String from the URL to the target type (Long, int, String, etc.). This is how RESTful resource-by-ID endpoints are implemented.

9

Write a simple method in a Spring Boot controller that accepts a JSON object in the request body and prints one of its fields.

Remember which annotation is used to bind the request body to a method parameter.

MODEL ANSWER

`@PostMapping("/events")` public ResponseEntity<String> createEvent(@RequestBody Event event) { System.out.println("Event name: " + event.getName()); eventService.save(event); return ResponseEntity.status(201).body("Created"); }. `@RequestBody` tells Spring to deserialize the incoming JSON into an Event object using Jackson. The client must send a Content-Type: application/json header. `ResponseEntity` lets you control the HTTP status code in the response — 201 Created is appropriate for a successful POST.

10

What is Spring Data JPA and how does it simplify database operations?

Focus on how it reduces the amount of code needed for CRUD operations.

MODEL ANSWER

Spring Data JPA is a layer on top of JPA (Hibernate) that auto-generates repository implementations at runtime. You define an interface extending `JpaRepository<Entity, ID>` and Spring creates a bean with full CRUD methods (save, findById, findAll, delete) without you writing any implementation code. You can declare query methods by naming convention — `'findByName(String name)'` generates the correct HQL automatically. This eliminates most DAO boilerplate while still giving you `@Query` for complex cases.

11

What is the purpose of the Entity, Repository, and Service layers in a typical Spring Boot application?

MODEL ANSWER

The Entity layer (`@Entity` classes) maps Java objects to database tables — each field corresponds to a column. The Repository layer (interfaces extending `JpaRepository`) handles all database interaction — Spring Data generates the implementation. The Service layer (`@Service` classes) contains business logic — it orchestrates multiple repository calls, applies rules, and handles transactions. This separation means your business logic is not tangled with SQL, your entities are not bloated with controller code, and each layer is independently testable and replaceable.

12

How does the CrudRepository interface help you perform database operations in Spring Boot?

MODEL ANSWER

CrudRepository provides ready-made methods: save(entity) for INSERT/UPDATE, findById(id) returning Optional<T>, findAll() returning Iterable<T>, deleteById(id), and count(). You get all of these by extending the interface with your entity type and ID type: 'interface EventRepository extends CrudRepository<Event, Long>'. No implementation class is needed — Spring Data creates it at startup. JpaRepository extends CrudRepository and adds paging/sorting support, making it the more commonly used option in Spring Boot projects.

13

Suppose you need to save an Event object to a MySQL database using Spring Data JPA. Briefly describe the steps required.

MODEL ANSWER

Add spring-boot-starter-data-jpa and mysql-connector-j to pom.xml. Configure application.properties with spring.datasource.url, username, password, and spring.jpa.hibernate.ddl-auto=update. Annotate the Event class with @Entity and mark the id field with @Id and @GeneratedValue. Create interface EventRepository extending JpaRepository<Event, Long>. Inject EventRepository into your service with @Autowired. Call 'eventRepository.save(event)' to persist. That is the complete setup — no SQL, no connection management, no DAO boilerplate.

14

How would you define a simple JPA entity class for an Event with fields: id, name, and date?

Consider which annotations are needed to mark a class and its fields as a JPA entity.

MODEL ANSWER

```
'@Entity @Table(name="events") public class Event { @Id
@GeneratedValue(strategy=GenerationType.IDENTITY) private Long id; @Column(nullable=false) private
String name; private LocalDate date; // getters and setters }'. @Entity tells JPA this class maps to a table.
@Table specifies the table name. @Id marks the primary key. @GeneratedValue(IDENTITY) delegates key
generation to the database auto-increment. @Column constraints are optional but improve clarity. Hibernate
will create the table automatically if ddl-auto=update.
```

15

What is Spring Security and why is it important in web applications?

Think about authentication and protecting resources.

MODEL ANSWER

Spring Security is a framework that handles authentication (who are you?) and authorisation (what are you allowed to do?) for Spring applications. Without it, any user can access any endpoint. Spring Security intercepts every request through a filter chain, checks credentials, and enforces access rules. It supports HTTP Basic, form login, JWT, and OAuth2 out of the box. It also protects against common vulnerabilities like CSRF, session fixation, and clickjacking automatically. Adding the spring-boot-starter-security dependency gives you a secured application immediately.

16

How does Basic Authentication work in a Spring Boot REST API?

Consider what information is sent with each request and how the server verifies it.

MODEL ANSWER

With Basic Authentication, the client sends the username and password encoded in Base64 in the Authorization header of every request: 'Authorization: Basic dXNlcjpwYXNz'. Spring Security's filter chain decodes the header, looks up the user in the configured UserDetailsService, and compares the password (usually against a BCrypt hash). If credentials are valid, the request proceeds; if not, a 401 Unauthorized response is returned. Basic Auth is simple but must always be used over HTTPS because Base64 is trivially decodable.

17

What does the @Valid annotation do when applied to a method parameter in a controller?

Think about input validation and error handling.

MODEL ANSWER

@Valid triggers Bean Validation on the annotated parameter before the method body runs. Spring checks all constraint annotations on the object's fields (@NotNull, @Size, @Email, etc.) and if any fail, throws a MethodArgumentNotValidException (or populates a BindingResult if one is present). This prevents invalid data from reaching the service or database. In a REST API, uncaught validation exceptions return a 400 Bad Request with details. Use @Valid on @RequestBody parameters for JSON input and on @ModelAttribute parameters for form data.

18

Give an example of how you would use @NotNull in a Spring Boot entity class.

MODEL ANSWER

```
@Entity public class Event { @Id @GeneratedValue private Long id; @NotNull(message="Event name is required") @Size(min=3, max=100, message="Name must be between 3 and 100 characters") private String name; @NotNull(message="Date is required") private LocalDate date; }
```

When you pass an Event with a null name to a controller method annotated @Valid, Spring rejects the request before it reaches the service. The message attribute provides the human-readable error description returned to the client.

19

If you want to protect the 'Add Event' API so only authenticated users can access it, what changes would you make in your Spring Boot project?

Think about configuration and annotations related to security.

MODEL ANSWER

Add spring-boot-starter-security to pom.xml. Create a class extending WebSecurityConfigurerAdapter (or a SecurityFilterChain @Bean in newer Spring Boot) and configure:

```
'http.authorizeRequests().antMatchers(HttpMethod.POST, "/events").authenticated().anyRequest().permitAll().and().httpBasic()'. Configure a UserDetailsService bean that loads user credentials from the database. Now POST /events requires a valid Authorization header; other endpoints remain open. Use BCryptPasswordEncoder to hash passwords when storing them.
```

20

How would you test a REST API endpoint created in Spring Boot using Postman?**MODEL ANSWER**

Open Postman and create a new request. Set the method (GET, POST, etc.) and URL (e.g., `http://localhost:8080/events`). For POST, set the body to 'raw', select 'JSON', and type the event JSON. If the endpoint is secured, add Basic Auth credentials in the Authorization tab. Click Send. Verify the response status code (200, 201, 400, etc.), the response body (correct JSON structure), and the response time. Use Postman Collections to group all API tests and run them together. The Environments feature lets you switch between localhost and production URLs.

21

What is CORS and why do you need to enable it when connecting a React frontend to your Spring Boot backend?*Consider how browsers handle requests from different origins.***MODEL ANSWER**

CORS (Cross-Origin Resource Sharing) is a browser security policy that blocks JavaScript from making requests to a different origin (different domain, port, or protocol) than the page it is loaded from. If your React app runs on `localhost:3000` and your Spring Boot API on `localhost:8080`, the browser blocks the request. To allow it, the Spring Boot server must send CORS headers (Access-Control-Allow-Origin) in the response. Without enabling CORS on the backend, even a correctly written React app will fail to fetch data from the API.

22

Suppose your React frontend fails to fetch data from the `/events` API due to a CORS error. What would you check and how would you fix it in your Spring Boot application?**MODEL ANSWER**

First, check the browser console — the error will say 'CORS policy: No Access-Control-Allow-Origin header'. Fix it by adding `@CrossOrigin(origins="http://localhost:3000")` to the controller class or method. For a global fix, configure a `WebMvcConfigurer` bean:

```
'registry.addMapping("/api/**").allowedOrigins("http://localhost:3000").allowedMethods("GET","POST","PUT",
"DELETE");
```

In production, replace `localhost:3000` with your actual frontend domain. If Spring Security is active, also configure CORS there, as the security filter chain intercepts before MVC.

23

Describe the steps to verify that your `/events` API is returning the correct data using Postman.**MODEL ANSWER**

Send GET `http://localhost:8080/events`. Confirm the response status is 200 OK. Inspect the response body — it should be a JSON array of event objects. Verify each object has the expected fields (id, name, date) with correct types. Check that the data matches what is in your database by querying MySQL directly and comparing. Test edge cases: GET `/events/{id}` for a non-existent ID should return 404, not 500. Test an authenticated endpoint by toggling auth headers on and off to confirm security is working. Save the requests in a Postman collection for reuse.

24

How would you structure your backend code to separate concerns between controllers, services, and repositories in a Spring Boot project?

Think about what each layer is responsible for.

MODEL ANSWER

Controllers (@RestController) receive HTTP requests, validate input, and delegate to the service — no business logic, no SQL. Services (@Service) contain business logic — input transformation, rules, calling multiple repositories, handling transactions. Repositories (JpaRepository interfaces) handle all database access — no business logic. Entity classes (@Entity) represent the data model. Package by layer (controller, service, repository, entity) or by feature (events, users). Each layer depends only on the layer below it through injected interfaces, never skipping layers.

25

In your mini project, how would you implement user registration and login with security in Spring Boot?

MODEL ANSWER

Registration: POST /auth/register accepts a User JSON, hashes the password with BCryptPasswordEncoder, saves via UserRepository, and returns 201. Login: configure Spring Security's form login or Basic Auth to authenticate against a custom UserDetailsService that loads the user from the database by email. On successful authentication, return a session cookie or JWT token. Annotate protected endpoints with @PreAuthorize or configure HttpSecurity to require authentication. Store only the BCrypt-hashed password — never the plain text.

26

Explain how you would perform CRUD operations for event management in your Campus Connect Portal backend.

MODEL ANSWER

GET /events — EventService calls eventRepository.findAll(); returns list as JSON. GET /events/{id} — eventRepository.findById(id) returns Optional; throw ResourceNotFoundException if empty. POST /events — validate @RequestBody Event, call eventRepository.save(event), return 201 with the saved event. PUT /events/{id} — find existing event, update fields from request body, call save(). DELETE /events/{id} — verify existence, call deleteById(). All methods are in EventController; delegation goes to EventService; persistence is through EventRepository. Each controller method has a matching Postman test.

27

How would you integrate your Spring Boot REST API with a React frontend using Axios?

MODEL ANSWER

In React, install Axios with 'npm install axios'. Make a GET call: 'axios.get("http://localhost:8080/events").then(res => setEvents(res.data))'. For POST: 'axios.post("/events", newEvent, { headers: { "Content-Type": "application/json" } })'. Enable CORS on the Spring Boot backend for the React origin. For authenticated endpoints, attach the auth header: 'axios.defaults.headers.common["Authorization"] = "Basic " + btoa(user+":"+pass)'. Use an Axios interceptor to add the token globally and handle 401 errors by redirecting to login.

28

What does it mean for your backend to be 'microservice-ready'? How would you design your Spring Boot application to support this?

Consider scalability, modularity, and separation of concerns.

MODEL ANSWER

A microservice-ready backend has no shared in-memory state between requests (it is stateless), stores session data externally (JWT or Redis), reads configuration from environment variables (not hardcoded), exposes health endpoints (Spring Actuator), and has loosely coupled modules. Design each feature as an independently deployable unit — UserService, EventService — connected only through REST APIs, not shared database tables. Use Spring Cloud dependencies (Eureka, Config Server) from the start so splitting the monolith into microservices later is straightforward.

29

What are some important steps to prepare your Spring Boot application for deployment on platforms like Heroku or Render?

MODEL ANSWER

Configure the port from an environment variable: 'server.port=\${PORT:8080}'. Externalise database credentials using environment variables or a config server — never hardcode in application.properties. Package as an executable JAR with 'mvn clean package'. Add a Procfile (Heroku) or configure the start command to 'java -jar target/myapp.jar'. Use a cloud database (PlanetScale, Amazon RDS) instead of localhost MySQL. Ensure Spring Boot's health endpoint is reachable so the platform can verify the app started. Set spring.jpa.hibernate.ddl-auto=validate or none in production.

30

How would you use Postman's AI features to automate API test case generation for your project?

Think about how AI can suggest or generate test cases based on your API schema.

MODEL ANSWER

In Postman, open a request and click the 'Tests' tab. Use Postman's AI (Postbot) to generate test scripts — it can suggest assertions like 'pm.response.to.have.status(200)', JSON schema validation, and response time checks based on the response it sees. For a collection, Postbot can generate a full test suite covering happy paths, error codes, and edge cases. Review each generated test to ensure it actually validates the business logic (not just the status code). Export the collection to run in Newman for CI/CD pipeline integration.

31

Suppose you want to automatically test all CRUD endpoints of your /events API. How can Postman collections and AI help streamline this process?

MODEL ANSWER

Create a Postman collection with requests for GET /events, POST /events, GET /events/{id}, PUT /events/{id}, and DELETE /events/{id}. Use Postbot to generate test assertions for each request. Chain them with Postman's Collection Runner — variables like 'eventId' can be set from the POST response and used in subsequent GET/PUT/DELETE requests. Run the collection in order to simulate the full CRUD lifecycle. Add the collection to a Newman command in your GitHub Actions pipeline so it runs after every deployment, catching regressions automatically.

32

How could you use ChatGPT or GitHub Copilot to help you debug a problem in your Spring Boot application, and what steps would you take to ensure the suggested solution is reliable?

Consider code review, testing, and understanding the AI's suggestions.

MODEL ANSWER

Paste the full stack trace and the relevant controller/service/entity code into ChatGPT and describe the expected vs. actual behaviour. ChatGPT will usually identify the root cause — a missing annotation, wrong method name, or misconfiguration — and suggest a fix. Copilot in the IDE can suggest code completions as you type the fix. Before using the suggestion: understand why the fix works (not just copy-paste), write a unit test that reproduces the bug and confirm it fails before the fix, apply the fix, and confirm the test now passes. Check the fix against Spring's documentation.

33

If you use an AI tool to generate boilerplate code for a new REST controller in Spring Boot, what should you always verify before using the code in your project?

MODEL ANSWER

Check that the annotations are correct for your Spring Boot version (@RestController, @RequestMapping, @Valid, etc.). Verify that input parameters use @RequestBody for JSON payloads and @PathVariable for path segments. Ensure all endpoints return appropriate HTTP status codes (201 for creation, 204 for deletion, 404 for not found). Confirm that no business logic is placed directly in the controller — it should delegate to a service. Check that exception handling is present (try-catch or @ControllerAdvice). Finally, run the application and test each endpoint with Postman to confirm the actual behaviour matches the generated code.

34

Integrative: How would you secure the POST /events endpoint using both Basic Authentication and input validation? Briefly outline the annotations and configuration you would use.

Think about combining security and validation features.

MODEL ANSWER

Configure HttpSecurity to require authentication for POST /events:

'http.authorizeRequests().antMatchers(HttpMethod.POST, "/events").authenticated().and().httpBasic()'. In the controller method: '@PostMapping("/events") public ResponseEntity<Event> create(@Valid @RequestBody Event event)'. Add @NotNull and @Size constraints to Event fields. Spring Security intercepts the request first — if no valid credentials, returns 401. If authenticated, Spring validates the body — if @Valid fails, returns 400. Only if both pass does the method execute. This is a two-layer defence: authentication at the security filter, validation at the method.

35

Integrative: Describe the flow of a request from a React frontend to a Spring Boot backend that fetches a list of events, mentioning CORS, the controller, service, repository, and database.

MODEL ANSWER

React sends 'axios.get("http://localhost:8080/events")'. The browser's preflight OPTIONS request checks CORS. Spring Boot's CORS config responds with Access-Control-Allow-Origin. The actual GET request arrives; DispatcherServlet routes it to EventController's @GetMapping method. The controller calls EventService.findAll(). The service calls EventRepository.findAll(), which Spring Data translates to 'SELECT * FROM events' and executes against MySQL. Results are mapped to a List<Event>. The controller returns the list; Spring's Jackson serialiser converts it to JSON. The React component receives the JSON array and renders the events.

36

Integrative: If a user is unable to add a new event due to a validation error, but is authenticated, how would you troubleshoot and resolve the issue in your Spring Boot application?

Consider both security and validation aspects.

MODEL ANSWER

Since the user is authenticated, security is not the issue — focus on validation. Check the request body in Postman: are all required fields present and correctly named? Enable Spring Boot debug logging ('logging.level.org.springframework.web=DEBUG') to see the full request and binding result. Add a @ControllerAdvice with a @ExceptionHandler(MethodArgumentNotValidException.class) that returns a structured error response listing which fields failed validation and why. Confirm the entity's constraint annotations (@NotNull, @Size) match what the client is sending. Fix either the client payload or relax the constraints if they are too strict.

37

Write a Spring Boot controller method that updates an event's name using its ID, and explain how you would test this endpoint using Postman.

MODEL ANSWER

@PutMapping("/events/{id}") public ResponseEntity<Event> updateEvent(@PathVariable Long id, @RequestBody Map<String,String> body) { Event event = eventService.findById(id).orElseThrow(() -> new RuntimeException(HttpStatus.NOT_FOUND)); event.setName(body.get("name")); return ResponseEntity.ok(eventService.save(event)); }. In Postman: set method to PUT, URL to /events/5, body to '{ "name": "New Name" }' (raw JSON). Send and verify: 200 with updated event, name changed. Test 404 by using a non-existent ID. Test 400 by sending an empty name if @NotNull is on the field.

38

Suppose your deployed Spring Boot application on Heroku is not connecting to the MySQL database. What steps would you take to diagnose and fix this issue?

MODEL ANSWER

Check Heroku logs with 'heroku logs --tail' — look for 'Connection refused' or authentication errors. Verify the DATABASE_URL or individual DB env vars (host, port, name, user, password) are correctly set in Heroku's config vars. Confirm the MySQL instance is accessible from Heroku's IP range (whitelisting may be needed). Check that the MySQL driver version in pom.xml is compatible with your database version. Ensure spring.datasource.url uses the correct format and that SSL parameters match ('?useSSL=false' or '?sslMode=REQUIRED' depending on the provider). Try connecting from your local machine using the same credentials to isolate the issue.

39

How does using Service and Repository layers make your Spring Boot application more maintainable and testable?

Think about separation of business logic and data access.

MODEL ANSWER

The Service layer can be tested without a database by injecting a mock Repository — unit tests are fast and reliable. The Repository layer is tested separately with an in-memory H2 database. Business logic changes stay inside the Service; data access changes stay in the Repository; neither bleeds into the Controller. If you switch from JPA to a different persistence mechanism, only the Repository changes — the Service and Controller are untouched. New developers can understand the codebase faster because each class has a clear, single responsibility.

40

What are some common mistakes freshers make when designing REST APIs in Spring Boot, and how would you avoid them?

MODEL ANSWER

Using wrong HTTP methods (GET for deletion, POST for read). Fix: follow REST conventions. Returning 200 OK for every response including errors. Fix: use appropriate status codes (201, 400, 404, 500). Exposing entity objects directly from the controller — this leaks database schema and causes circular serialisation. Fix: use DTOs. Hardcoding credentials in application.properties. Fix: use environment variables. Not handling exceptions — unhandled errors expose stack traces to clients. Fix: implement a `@ControllerAdvice` global handler. Missing validation — allowing empty or null fields. Fix: add `@Valid` and constraint annotations.